Lab 6: Code and Branch Coverage Analysis

Fuel Station - `fuel_station.py`

Name	SRN
Rohith Karning	PES1UG24AM229

1. Objective

Measure the code (line) and branch coverage of the starter tests for `fuel_station.py`, find the untested lines and branches, and write extra test cases to reach at least 90% code coverage and at least 90% branch coverage.

2. Module Under Test

Function	What it does	Decisions
fuel_bill(grade, litres, loyalty_tier)	Bill for litres above the free voucher litres. Gold gets 5 bonus litres on petrol; over 30 extra litres gets a bulk discount, more for gold.	7 if statements (unknown grade, litres < 0, gold bonus, extra <= 0, diesel rate, extra > 30, gold discount)
pump_allowed(litres, containers, is_fleet)	Fleet can draw up to 200 L. Others up to 60 L and at most 1 container.	is_fleet, litres > 60, containers <= 1
session_status(seconds_left, is_card, is_night)	Cutoff 60 s for card, 30 s for cash, +30 s at night. Active, warning (within 15 s of cutoff) or timed_out.	card/cash, night, active / warning / timed_out

3. Commands Used

```
pip install coverage pytest

coverage run -m pytest CodeCoverageTest_fuel_station.py

coverage report -m

coverage run --branch -m pytest BranchCoverageTest_fuel_station.py

coverage report -m
```

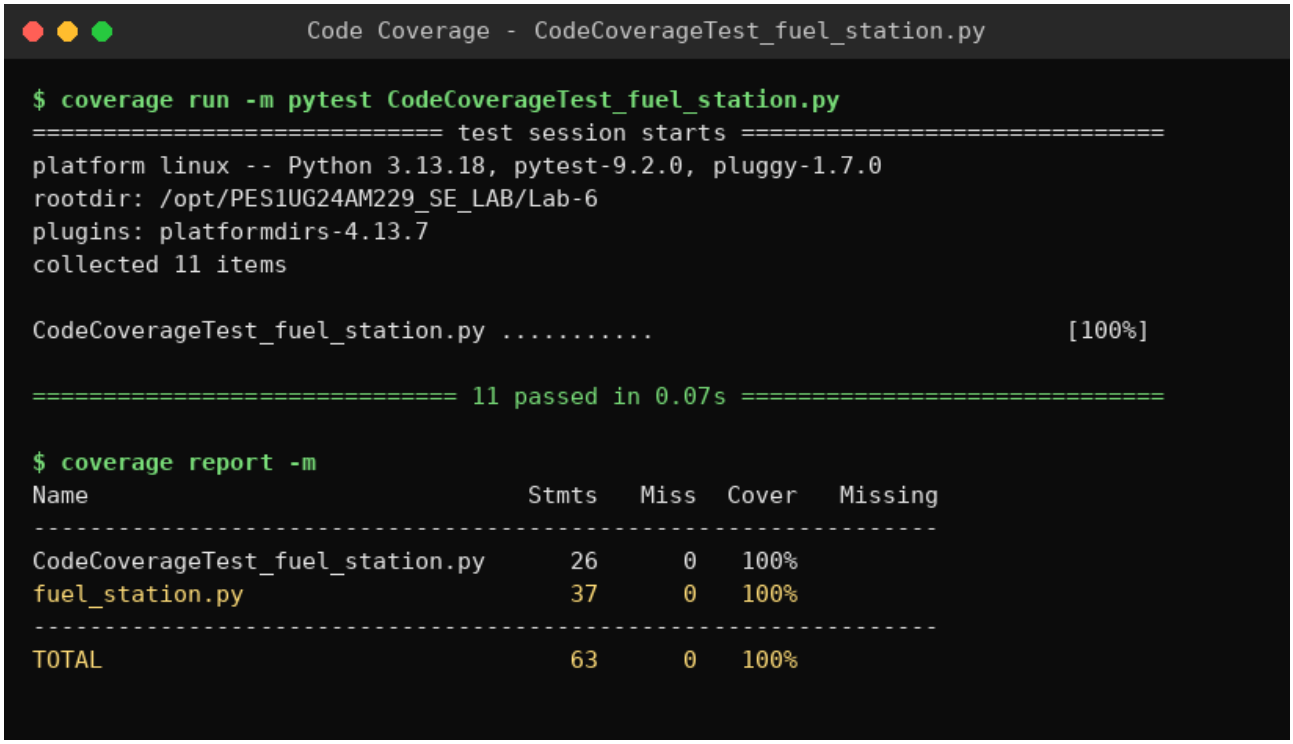
4. Results Summary

Test file	Tests	Line coverage	Branch coverage	Missing
test_fuel_station.py (starter)	2	38%	32%	6, 8, 11, 15-23, 28, 30, 35-45
CodeCoverageTest_fuel_station.py	11	100%	98%	branch 21->23
BranchCoverageTest_fuel_station.py	35	100%	100%	none

Percentages are for `fuel_station.py`. The starter tests only checked one free fill and one allowed pump request, so the bill calculation, fleet rules and the whole of `session_status` never ran.

5. Code (Line) Coverage Analysis

`CodeCoverageTest_fuel_station.py` has 11 tests, chosen so that every line runs at least once: an unknown grade and negative litres (the two ValueErrors), a free fill, a small diesel bill (diesel rate), a 60 L gold premium bill (gold bonus, bulk discount and gold discount lines), a fleet request, a non-fleet request over 60 L, and one session in each of the active, warning and timed_out states with both card and cash payments.

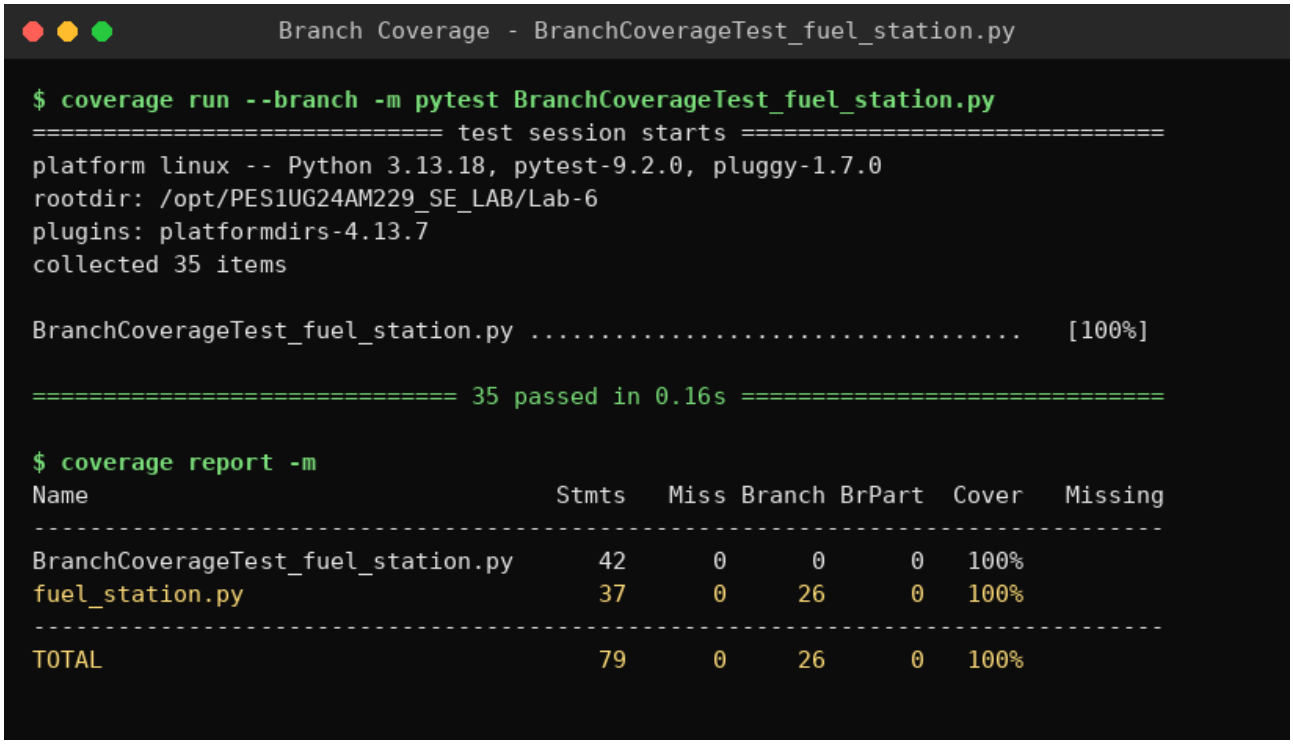


Line coverage is 100%, but when the same tests are run with `--branch` the result is 98% with branch **21->23** missing. The only test with more than 30 extra litres was a gold customer, so `if loyalty_tier == "gold"` inside the bulk discount was never False. Every line ran, but one path (bulk discount for a non-gold customer) was never tested. This is why 100% line coverage does not mean the code is fully tested.

6. Branch Coverage Analysis

`BranchCoverageTest_fuel_station.py` has 35 tests (using `pytest.mark.parametrize`) that make each condition go both ways and also check boundary values:

Decision	Test inputs (True / False sides and boundaries)
grade not in VOUCHER_LITRES	"kerosene" (raises) / regular, premium, diesel
litres < 0	-1 (raises) / 0 litres
gold AND not diesel	regular gold 15 L and 16 L / diesel gold, silver
extra <= 0	premium 15 L (extra = 0) / regular 11 L (extra = 1)
grade == diesel	diesel 25, 30, 60 L / regular and premium
extra > 30	regular 41 L (extra 31) / regular 40 L (extra exactly 30)
gold (bulk discount)	premium gold 60 L, diesel gold 60 L / regular and diesel silver (the branch the code coverage tests missed)
is_fleet, litres <= 200	200 L allowed, 201 L refused
litres > 60	61 L refused / 60 L allowed
containers <= 1	0 and 1 allowed / 2 refused
is_card_payment, is_night	all four combinations (cutoff 30, 60, 60, 90)
active / warning / timed_out	for each cutoff: exactly at cutoff, cutoff-1, cutoff-15 and cutoff-16



All 35 tests pass, and `fuel_station.py` has 100% statement coverage and 100% branch coverage (26 of 26 branches, 0 partial).

7. Conclusion

Coverage went from 38% line / 32% branch with the starter tests to 100% line coverage with the code coverage tests and 100% branch coverage with the branch coverage tests, both above the 90% target. Line coverage showed which lines had never run, but only branch coverage exposed the untested non-gold bulk discount path. Testing the boundary values (exactly 0 extra litres, exactly 30 extra litres, 60 and 61 litres, 200 and 201 litres, and the exact session cutoffs) also checks that each comparison uses the correct operator, such as >= rather than >.